

Toolbox-beginner-Rstudio

Natalia Galarza

2026-08-18

R Markdown

Hi! I am doing this cheat-sheet for myself so I can organize and recall all the info I have gotten from some R and Rstudio introductory courses. I am not a data scientist, I don't think like one. All this information won't just easily stick in my brain, nor is it obvious for me which command to use each time. Anyway, my purpose is to make a useful list of all these bureaucratic, dull commands.

R packages

Quick assumption before starting: Git and GitHub are already configured in RStudio.

Part 1

Install a package from CRAN

```
install.packages("packagename")
```

Install multiples packages from CRAN

```
install.packages(c("packagename", "packagename2", "packagename3"))
```

Install a package from GitHub

```
install.packages("pak")  
pak::pkg_install("author/package")
```

Install a package from bioconductor

```
install.packages("BiocManager")  
BiocManager::install("packagename")
```

Note: important for a bioinformatician, I guess.../>_<

Part 2: after installing a package you need to load it, which just means opening it.

Load a package

```
library(packagename)
```

Note: never put package name in quotes here.

Alternatively - 4th panel: *packages > check the box of packagename*

Part 3: commands useful to know things about the stuff you just did before. all of the following probably come in a sequence when executing.

Check installed packages

```
installed.packages()  
or  
library()
```

Note: if you are not sure if you have already installed a package, use either of these codes, with nothing between the parentheses to check.

Alternatively - 4th panel: *packages > view the list*

Check packages that need updates

```
old.packages()
```

Note: checks which installed packages have newer versions available, but it does not update them.

Update installed packages

```
update.packages()
```

Note: checks your installed packages and lets you update the ones that have newer versions available, like in one go you can update all the packages you just checked that need a package with the previous command just before this one >.<.

Update a specific package

```
install.packages("packagename")
```

Part 4: reversing stuff.

Unload a package

```
detach("package:ggplot2", unload = TRUE)
```

Note: removes the loaded package from the current R session.

Alternatively - 4th panel: *packages* > *uncheck box of packagename*

Uninstall a package

```
remove.packages("ggplot2")
```

Alternatively - 4th panel: *Packages* > *click X on packagename*

Part 5: other few things you might need to know, mainly for better understanding what you've got.

Check session information

```
sessionInfo()
```

Note: shows the R version, operating system and packages loaded in the current session.

View package help

```
help(package = "packagename")
```

View package vignettes

```
browseVignettes()  
browseVignettes("packagename")
```

Note: vignettes are longer guides or tutorials created to explain how to use a package and its functions. the first command will show vignettes available from installed packages, while the second command shows only the vignettes available for your specific package.

Projects

About creating and opening

Creating a project

1. New directory

file > new project > new directory > new project

2. Existing directory

As far as I understand, you use this when you already have a folder locally with files you want to work with, and you want to turn that folder into an RStudio project.

file > new project > existing directory > select existing folder > create project

Note: this only creates the RStudio Project inside the existing folder, it does not automatically mean that the project is under Git or GitHub version control.

If you also want to put this existing project under Git version control:

First, open Git Bash and navigate to the project folder

```
cd ~/c/Users/Nataliaa/Desktop/projectname
```

Then initialize Git inside that folder

```
git init
```

Note: this creates a local Git repository inside the current project folder.

Then stage all files and changes inside the current directory

```
git add .
```

Then create the first commit

```
git commit -m "Initial commit"
```

Note: -m refers to the commit message, so inside the quotation marks you can write the message you want for that commit.

Up to this point, the project is under local Git version control only.

If you also want to link this local project to GitHub

open GitHub > create new repo and give it the same name as the R project > copy the repo URL

Then

open Git > run the commands provided at the very end of your chosen repo (... or push an existing repository from the command line)

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
```

Then push your local project to GitHub for the first time

```
git push -u origin main
```

Note: -u establishes origin/main as the upstream branch. After this first push, future pushes can normally be done simply by pushing on Rstudio or with the 'git push' command.

3. Version Control

As far as I understand, this is the same as **cloning**, meaning that you use this when you've created a repo on Github first or you are forking someone else's repo, and you want to clone the repo into your computer and open it as an RStudio project.

file > new project > version control > Git

Then:

copy and paste repository URL from GitHub > select local location > create project

Note: again, this clones an existing git repository from github to your computer and opens it as an RStudio project.

Opening a project

1. Double click on 'Rproj' on your computer
2. *file > open project*
3. *project toolbar (upper-right) > open project*

About closing and switching

Closing a project

1. *file > close project*
2. *project toolbar (upper-right) > close project*

Switching to another project

Closing current project and opening another project in the same window:

project toolbar (upper-right) > open project

Without closing the current one:

project toolbar (upper-right) > open project in new session

GitHub and Git

Git configuration

```
git config --list
```

Note: shows the current git configuration, meaning basically username and email.

If you want to change this, then:

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```